

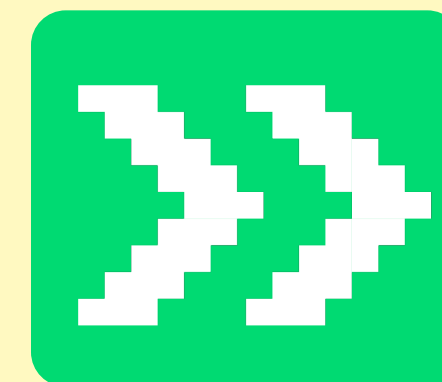
Как проектировать HTTP API

YOUNG & YANDEX

Александр Куваев

Руководитель службы разработки
клиентского бэкенда и данных в
Яндекс Лавке

Руководитель C++ трека ШБР



О чем эта лекция

01



HTTP протокол

- Структура сообщения
- Методы
- Заголовки
- Тело
- Коды ответов

02



API best practice

- Качество API
- Идемпотентность
- Атомарность
- Обратная совместимость
- Версионирование
- Best practice при проектировании API

03



REST API

- Понятие REST
- Модель зрелости REST-сервисов

04



RPC

- Понятие RPC
- Json-RPC
- SOAP
- gRPC

Структура HTTP сообщения

Request



Response



Стартовая строка

Request



```
$ curl -v https://ya.ru/ping
> GET /ping HTTP/2
...
>
< HTTP/2 404
...
```

Строка статуса

```
$ curl -v https://ya.ru/ping
> GET /ping HTTP/2
...
>
< HTTP/2 404
...
```

Response

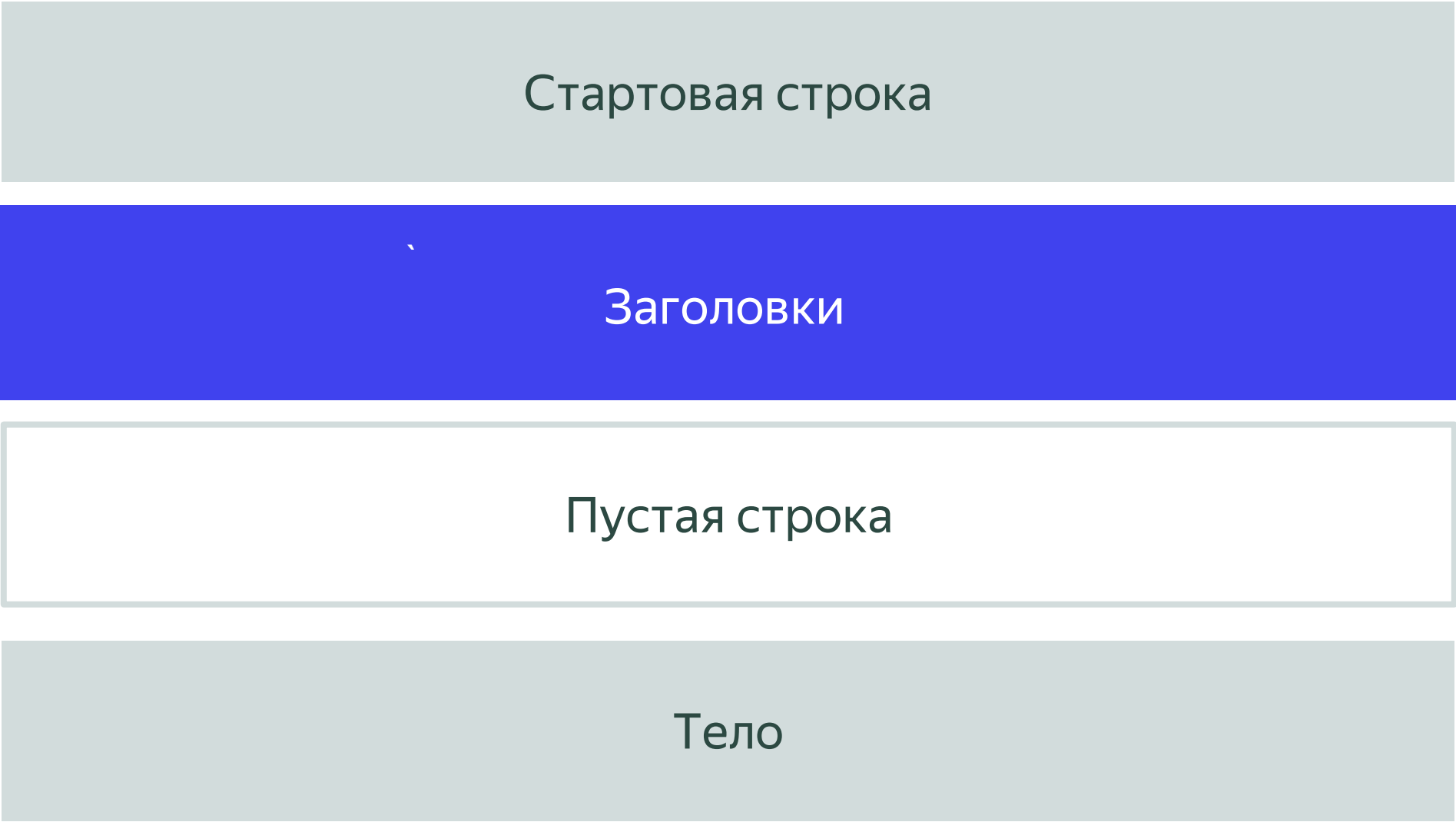
Строка статуса
Заголовки
Пустая строка
Тело

Методы HTTP

Метод	Тело запроса	Тело ответа	Идемпотентен	Read-only
CONNECT	Не рекомендуется	Есть	Нет	Нет
DELETE	Не рекомендуется	Есть	Да	Нет
GET	Не рекомендуется	Есть	Да	Да
HEAD	Не рекомендуется	Нет	Да	Да
OPTIONS	Не рекомендуется	Есть	Да	Да
PATCH	Есть	Есть	Нет	Нет
POST	Есть	Есть	Нет	Нет
PUT	Есть	Есть	Да	Нет
TRACE	Нет	Есть	Да	Да

HTTP заголовки

Request



Response



Заголовки запроса и ответа

<https://ya.ru>



Заголовки запроса

accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9

accept-encoding: gzip, deflate, br

accept-language: ru,en;q=0.9,is;q=0.8

Заголовки запроса

accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
accept-encoding: gzip, deflate, br
accept-language: ru,en;q=0.9,is;q=0.8
cache-control: no-cache

Заголовки запроса

accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
accept-encoding: gzip, deflate, br
accept-language: ru,en;q=0.9,is;q=0.8
cache-control: no-cache
cookie: SECRET

Заголовки запроса

accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
accept-encoding: gzip, deflate, br
accept-language: ru,en;q=0.9,is;q=0.8
cache-control: no-cache
cookie: SECRET
referer: https://yandex.ru/

Заголовки запроса

accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
accept-encoding: gzip, deflate, br
accept-language: ru,en;q=0.9,is;q=0.8
cache-control: no-cache
cookie: SECRET
referer: https://yandex.ru/
user-agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 YaBrowser/23.3.1.840 (beta) Yowser/2.5 Safari/537.36

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Downlink, ECT

accept-ch-lifetime: 31536000

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Downlink, ECT

accept-ch-lifetime: 31536000

cache-control: no-cache,no-store,max-age=0,must-revalidate

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Dnlink, ECT

accept-ch-lifetime: 31536000

cache-control: no-cache,no-store,max-age=0,must-revalidate

content-encoding: br

content-type: text/html; charset=UTF-8

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Downlink, ECT

accept-ch-lifetime: 31536000

cache-control: no-cache,no-store,max-age=0,must-revalidate

content-encoding: br

content-type: text/html; charset=UTF-8

date: Tue, 17 May 2023 19:28:36 GMT

expires: Tue, 17 May 2023 19:28:37 GMT

last-modified: Tue, 17 May 2022 19:28:37 GMT

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Downlink, ECT

accept-ch-lifetime: 31536000

cache-control: no-cache,no-store,max-age=0,must-revalidate

content-encoding: br

content-type: text/html; charset=UTF-8

date: Tue, 17 May 2023 19:28:36 GMT

expires: Tue, 17 May 2023 19:28:37 GMT

last-modified: Tue, 17 May 2023 19:28:37 GMT

set-cookie: SECRET

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Downlink, ECT

accept-ch-lifetime: 31536000

cache-control: no-cache,no-store,max-age=0,must-revalidate

content-encoding: br

content-type: text/html; charset=UTF-8

date: Tue, 17 May 2023 19:28:36 GMT

expires: Tue, 17 May 2023 19:28:37 GMT

last-modified: Tue, 17 May 2023 19:28:37 GMT

set-cookie: SECRET

status: 200

Заголовки ответа

accept-ch: Viewport-Width, DPR, Device-Memory, RTT, Downlink, ECT

accept-ch-lifetime: 31536000

cache-control: no-cache,no-store,max-age=0,must-revalidate

content-encoding: br

content-type: text/html; charset=UTF-8

date: Tue, 17 May 2023 19:28:36 GMT

expires: Tue, 17 May 2023 19:28:37 GMT

last-modified: Tue, 17 May 2023 19:28:37 GMT

set-cookie: SECRET

status: 200

x-requestid: 1664479716471331-3381410595086230351-vla1-0467-vla-l7-balancer-8080-BAL-9691

Тело запроса

Request

Стартовая строка
Заголовки
Пустая строка
Тело

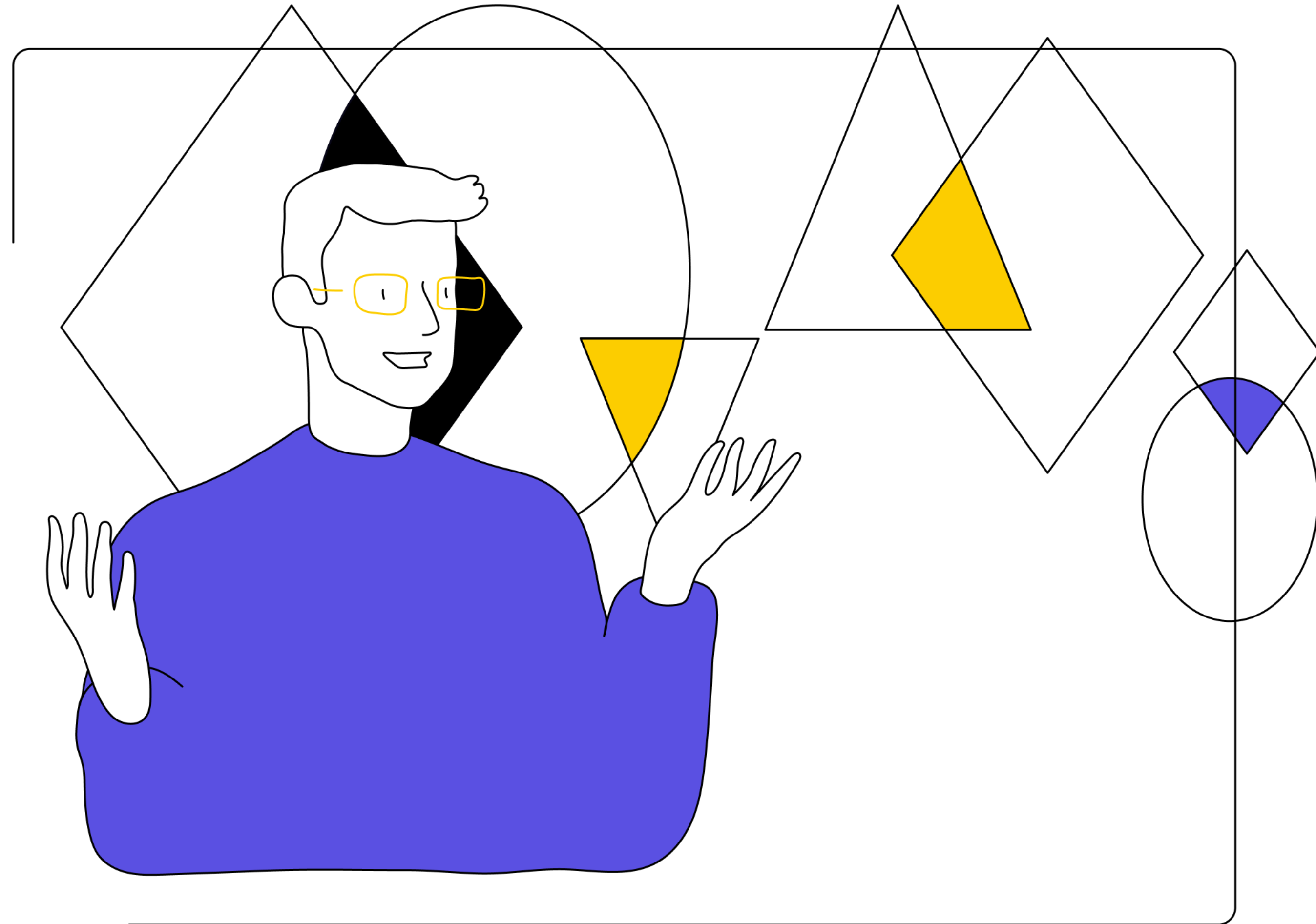
Response

Строка статуса
Заголовки
Пустая строка
Тело

Коды ответов

Основные типы

- 1xx – информационные
- 2xx – успех
- 3xx – перенаправление
- 4xx – ошибка клиента
- 5xx – ошибка сервера



Успех

```
$ curl -v https://lavka.yandex.ru
```

```
> GET / HTTP/2
```

```
> Host: lavka.yandex.ru
```

```
> User-Agent: curl/7.58.0
```

```
> Accept: */*
```

```
>
```

```
< HTTP/2 200
```

```
< x-yatraceid: 9e654367-9b3d-46c8-90ef-5c203dae1282
```

```
< cache-control: max-age=0, must-revalidate, no-cache, no-store, private, proxy-revalidate
```

```
< date: Fri, 17 May 1991 20:18:00 GMT
```

```
< set-cookie: SECRET
```

```
< content-type: text/html; charset=utf-8
```

```
< x-yarequestid: c9eb5efc-da0d-4fcd-99bf-cb9644805283
```

```
<
```

```
<!DOCTYPE html><html...
```

Перенаправление

```
$ curl -v https://url_shortener.ru/ad67befb-da91-4c3b-8a92-957b49261594
```

```
> Host: yandex.ru
```

```
> User-Agent: curl/7.58.0
```

```
> Accept: */*
```

```
>
```

```
< HTTP/2 302
```

```
< location: https://example.com/
```

```
< date: Sat, 17 May 2032 20:14:21 GMT
```

```
< x-yandex-req-id: 1664482461504836-18213635360057874106-vla1-4519-vla-l7-balancer-8080-BAL-9698
```

```
< cache-control: max-age=1209600,private
```

```
<
```

Ошибка клиента

```
$ curl -v https://ya.ru/ping
```

```
> GET /ping HTTP/2
```

```
> Host: ya.ru
```

```
> User-Agent: curl/7.58.0
```

```
> Accept: */*
```

```
>
```

```
< HTTP/2 404
```

```
< date: Tue, 17 May 2023 20:54:44 GMT
```

```
< expires: Tue, 17 May 2023 20:54:45 GMT
```

```
< x-xss-protection: 1; mode=block
```

```
< last-modified: Tue, 17 May 2023 20:54:45 GMT
```

```
< cache-control: no-cache,no-store,max-age=0,must-revalidate
```

Ошибка сервера

```
$ curl -v localhost/ping
> GET /ping HTTP/1.1
> Host: localhost
> User-Agent: curl/7.58.0
> Accept: */*
>
< HTTP/1.1 502 Bad Gateway
< Server: nginx/1.14.2
< Date: Thu, 17 May 2001 20:30:24 GMT
< Content-Type: text/html
< Content-Length: 1677
< Connection: keep-alive
<
<!DOCTYPE html...
```

Вопросы?



Критерии качества API

- Структурная целостность

Критерии качества API

- Структурная целостность
- Очевидность

Критерии качества API

- Структурная целостность
- Очевидность
- Консистентность

Критерии качества API

- Структурная целостность
- Очевидность
- Консистентность
- Читаемость

Критерии качества API

- Структурная целостность
- Очевидность
- Консистентность
- Читаемость
- Устойчивость к изменениям

Структура и иерархия

- Слабая связность разных сущностей

Структура и иерархия

- Слабая связность разных сущностей
- Предоставление высокоуровневых абстракций

Структура и иерархия

- Слабая связность разных сущностей
- Предоставление высокоуровневых абстракций
- Изоляция уровней абстракции, взаимодействие только с соседними слоями

Структура и иерархия

- Слабая связность разных сущностей
- Предоставление высокоуровневых абстракций
- Изоляция уровней абстракции, взаимодействие только с соседними слоями
- Разграничение областей ответственности

Именование эндпоинтов

Из записи METHOD + url должно быть понятно, что делает операция

Плохо

POST /order/status
PUT /order
PATCH /order
GET /order/cancellation

Хорошо

POST /order/cancel
GET /order/cancellation-status
GET /order/status

Именованние эндпоинтов

Из записи METHOD + url должно быть понятно, что делает операция

Плохо

POST /order/status
PUT /order
PATCH /order
GET /order/cancellation

Хорошо

POST /order/cancel
GET /order/cancellation-status
GET /order/status

Если операция модифицирующая, то это должно быть очевидно из сигнатуры

Плохо

GET /order/cancel
GET /cart/create
GET /order/{order-id}/remove

Хорошо

POST /order/cancel
POST /cart/create
DELETE /order/{order-id}
GET /cart/{cart-id}
GET /order/{order-id}
GET /products

Допустимо

POST /cart/retrieve
POST /products-list

Именование сущностей

- Называйте сущности по-английски

Плохо

парка
zakaz
korzina

Хорошо

folder
order
cart

Именование сущностей

- Называйте сущности по-английски
- Не используйте одиночные слова без определённой семантики

Плохо

get
set
call
apply

Хорошо

get_id
set_date
call_timestamp
apply_discount

Именование сущностей

- Называйте сущности по-английски
- Не используйте одиночные слова без определённой семантики
- Не используйте сокращения (за исключением устоявшихся)

Плохо

l10n (localization)
l6n (location)
icnt (items_count)
apl_dcnt (apply_discount)
и прочие упрлс

Допустимо

str
tpl
ts
ms

Именование сущностей

- Называйте сущности по-английски
- Не используйте одиночные слова без определённой семантики
- Не используйте сокращения (за исключением устоявшихся)
- Используйте парные сущности консистентно

Плохо

start + end
begin + stop

Хорошо

start + stop
begin + end
left + right

Именование сущностей

- Называйте сущности по-английски
- Не используйте одиночные слова без определённой семантики
- Не используйте сокращения (за исключением устоявшихся)
- Используйте парные сущности консистентно
- Добавляйте префикс `is`, `has` к булевым сущностям

Плохо

```
discount: true  
product: true
```

Хорошо

```
has_discount: true  
is_product: true
```

Именование сущностей

- Называйте сущности по-английски
- Не используйте одиночные слова без определённой семантики
- Не используйте сокращения (за исключением устоявшихся)
- Используйте парные сущности консистентно
- Добавляйте префикс `is`, `has` к булевым сущностям
- Добавляйте множественное число к массивам

Плохо

`product: []`

Хорошо

`products: []`
`products-list: []`

Именование сущностей

- Называйте сущности по-английски
- Не используйте одиночные слова без определённой семантики
- Не используйте сокращения (за исключением устоявшихся)
- Используйте парные сущности консистентно
- Добавляйте префикс is, has к булевым сущностям
- Добавляйте множественное число к массивам
- Избегайте двойных отрицаний

Плохо

checkout_disabled: false
do_not_disturb: false

Хорошо

checkout_enabled: true/false
disturbing_is_allowed: true/false

Указывайте формат

- Явно прописывайте формат в описании API

```
type: integer  
format: int64
```

```
type: string  
format: date-time
```

```
type: string  
pattern: '^S{3}$'
```

Указывайте формат

- Явно прописывайте формат в описании API
- Можно указывать формат в названии поля

country_iso3
duration_ms

Указывайте формат

- Явно прописывайте формат в описании API
- Можно указывать формат в названии поля
- Можно разворачивать поле в объект

```
"duration": {  
  "data_type": "ms",  
  "value": 1000  
}
```

Не передавайте цены в number

- Используйте числа с фиксированной запятой там, где это возможно (*в Json невозможно*)

Не передавайте цены в number

- Используйте числа с фиксированной запятой там, где это возможно (*в Json невозможно*)
- При невозможности стоит передавать такие данные либо в виде целых чисел, либо в виде строк

price = 0.1 + 0.2 (= 0.30000000000000004)

Integer:

```
{  
  "price": 3  
  "divisor": 10  
}
```

string:

```
{  
  "price": "0.3"  
}
```

Не передавайте цены в number

- Используйте числа с фиксированной запятой там, где это возможно (*в Json невозможно*)
- При невозможности стоит передавать такие данные либо в виде целых чисел, либо в виде строк

price = 0.1 + 0.2 (= 0.30000000000000004)

Integer:

```
{  
  "price": 3  
  "divisor": 10  
}
```

string:

```
{  
  "price": "0.3"  
}
```

- В самом крайнем случае – опишите в документации принципы округления

Идентификаторы

- Используйте глобально уникальные идентификаторы (например uuid)

item_id = 4576c706-427c-11ed-b878-0242ac120002

user_id = baf748a6-93cf-4698-9c57-742888205c5a

order_id = 4c1755fa-0666-46cf-8062-f12380822d13

Идентификаторы

- Используйте глобально уникальные идентификаторы (например uuid)
- Используйте пространства имён, если нужно хранить разные сущности и уметь их отличать по идентификатору

item_id = item:4576c706-427c-11ed-b878-0242ac120002

user_id = user:baf748a6-93cf-4698-9c57-742888205c5a

order_id = order:4c1755fa-0666-46cf-8062-f12380822d13

Идентификаторы

- Используйте глобально уникальные идентификаторы (например uuid)
- Используйте пространства имён, если нужно хранить разные сущности и уметь их отличать по идентификатору
- **Не используйте автоинкремент**

8:00

POST /order/create

```
{  
  "order_id": 1000  
}
```

22:00

POST /order/create

```
{  
  "order_id": 2000  
}
```

Вопросы?



Пустой ответ не всегда 404

- 4xx код означает ошибку клиента, правда ли, что пустой ответ означает ошибку?

Пустой ответ не всегда 404

- 4xx код означает ошибку клиента, правда ли, что пустой ответ означает ошибку?
- Если клиент запрашивал массив опций, то является ли пустой массив ошибочным результатом?

Пустой ответ не всегда 404

- 4xx код означает ошибку клиента, правда ли, что пустой ответ означает ошибку?
- Если клиент запрашивал массив опций, то является ли пустой массив ошибочным результатом?
- Возникла ли какая-то нештатная ситуация в процессе обработки запроса?

Идемпотентность

Идемпотентные “Из коробки”

- GET
- DELETE
- PUT

Идемпотентность

Токен идемпотентности

POST /order/create

```
{
  "user_location": ...,
  "cart_info": ...,
  "idempotency_token": "9c9077ab-e85c-4ce4-8198-1e5fc2193f78"
  ...
} → 200 (Создан новый заказ)
{
  "order_id": "order:b5fd01d6-e130-463a-acc0-3b2b082635d6"
}
```

Идемпотентность

Токен идемпотентности

POST /order/create

```
{
  "user_location": ...,
  "cart_info": ...,
  "idempotency_token": "9c9077ab-e85c-4ce4-8198-1e5fc2193f78"
  ...
} → 200 (Создан новый заказ)
{
  "order_id": "order:b5fd01d6-e130-463a-acc0-3b2b082635d6"
}
```

POST /order/create

```
{
  "user_location": ...,
  "cart_info": ...,
  "idempotency_token": "9c9077ab-e85c-4ce4-8198-1e5fc2193f78"
  ...
} → 200 (Новый заказ не создан)
{
  "order_id": "order:b5fd01d6-e130-463a-acc0-3b2b082635d6"
}
```

Идемпотентность

Драфты

POST /order/create

```
{
  "user_location": ...,
  "cart_info": ...,
  ...
} → 200 (Создан драфт заказа)
{
  "draft_id": "draft:b5fd01d6-e130-463a-acc0-3b2b082635d6"
}
```

Идемпотентность

Драфты

POST /order/create

```
{
  "user_location": ...,
  "cart_info": ...,
  ...
} → 200 (Создан драфт заказа)
{
  "draft_id": "draft:b5fd01d6-e130-463a-acc0-3b2b082635d6"
}
```

PUT /order/confirm

```
{
  "draft_id": "draft:b5fd01d6-e130-463a-acc0-3b2b082635d6"
} → 200 (Создан новый заказ)
{
  "order_id": "order:b5fd01d6-e130-463a-acc0-3b2b082635d6"
}
```

Атомарность

Что делать, если часть данных
корректна, а часть нет?

Атомарность

POST /cart/update

```
{
  "cart_id": ...,
  "items": [
    {
      "id": "item-id-1",
      ...
    },
    {
      "id": "item-id-2",
      ...
    }
  ]
}
```

200

```
{
  "items": [
    {
      "id": "item-id-1",
      "status": "SUCCESS"
    },
    {
      "id": "item-id-2",
      "status": "PRICE_MISMATCH"
    }
  ]
}
```

Неатомарность плохо сочетается с
идемпотентностью

Машиночитаемые ошибки

Если в ошибке нет машиночитаемой информации –
лучшее, что может сделать клиент – показать её
пользователю

POST /order/create

409

```
{  
  "message": "product out of stock"  
}
```

409

```
{  
  "message": "price mismatch"  
}
```

Машиночитаемые ошибки

Разделяйте машиночитаемое сообщение и локализованное для пользователя

POST /order/create

409

```
{  
  "message": "item out of stock",  
  "code": "ITEM_OUT_OFF_STOCK"  
}
```

409

```
{  
  "message": "price mismatch",  
  "code": "ITEM_PRICE_MISMATCH"  
}
```

Машиночитаемые ошибки

Отлично, если сервис вернёт необходимую информацию для исправления ошибки

POST /order/create

409

```
{  
  "message": "item out of stock",  
  "code": "ITEM_OUT_OFF_STOCK",  
  "diff": {  
    "item_id": "item-id-1",  
    "cart_stock": 7,  
    "store_stock": 5  
  }  
}
```

409

```
{  
  "message": "price mismatch",  
  "code": "ITEM_PRICE_MISMATCH",  
  "diff": {  
    "item_id": "item-id-1",  
    "cart_price": "20.5",  
    "store_price": "15.1"  
  }  
}
```

Машиночитаемые ошибки

Идеально, если сервис вернёт сразу все ошибки, которые нужно починить, а не только первую найденную

POST /order/create

409

```
{
  "errors": [
    {
      "message": "item out of stock",
      "code": "ITEM_OUT_OFF_STOCK",
      "diff": {
        "item_id": "item-id-1",
        "cart_stock": 7,
        "store_stock": 5
      }
    },
```

```
{
  "message": "price mismatch",
  "code": "ITEM_PRICE_MISMATCH"
  "diff": {
    "item_id": "item-id-1",
    "cart_price": "20.5",
    "store_price": "15.1"
  }
}
] // errors
}
```

Порядок ошибок

- Если нет возможности вернуть сразу все ошибки – соблюдайте порядок

Порядок ошибок

- Если нет возможности вернуть сразу все ошибки – соблюдайте порядок
- Всегда показывайте неразрешимые ошибки, если они есть (например ограничение доступа)

Порядок ошибок

- Если нет возможности вернуть сразу все ошибки – соблюдайте порядок
- Всегда показывайте неразрешимые ошибки, если они есть (например ограничение доступа)
- Сначала стоит вернуть критичные ошибки, затем менее значимые

Порядок ошибок

- Если нет возможности вернуть сразу все ошибки – соблюдайте порядок
- Всегда показывайте неразрешимые ошибки, если они есть (например ограничение доступа)
- Сначала стоит вернуть критичные ошибки, затем менее значимые
- Если решение одних ошибок может приводить к повторному появлению уже решенных – нужно изменить схему так, чтобы не было повторений

Ограничения доступа

- Предусмотрите коды ошибок для ограничения трафика, доступа или перенаправлений на капчу
- Не обязательно сразу имплементировать их, но если описать их сразу, то добавление в будущем не станет разрывать версию API

401:

description: Unauthorized

content:

...

403:

description: Forbidden

content:

...

429:

description: Too Many Requests

content:

...

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос
- Удаление обязательных хедеров, полей из ответа

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос
- Удаление обязательных хедеров, полей из ответа
- Добавление в `enum` новых значений

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос
- Удаление обязательных хедеров, полей из ответа
- Добавление в enum новых значений
- **Добавление новых кодов ответов**

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос
- Удаление обязательных хедеров, полей из ответа
- Добавление в enum новых значений
- Добавление новых кодов ответов
- Изменение типов или форматов полей

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос
- Удаление обязательных хедеров, полей из ответа
- Добавление в enum новых значений
- Добавление новых кодов ответов
- Изменение типов или форматов полей
- Изменение метода у эндпоинта

Обратно несовместимые изменения

- Добавление новых обязательных параметров, полей, хедеров в запрос
- Удаление обязательных хедеров, полей из ответа
- Добавление в enum новых значений
- Добавление новых кодов ответов
- Изменение типов или форматов полей
- Изменение метода у эндпоинта
- И другие действия, которые могут приводить к ошибкам на клиентах

Версионирование

Версия API – изменяется при обратно несовместимых изменениях на уровне API (появились новые сущности, доступные только в v2 api или пропали старые)

`/v1/orders/v1/create`

`/v1/cart/v1/update`

`/v2/orders/v1/create`

Версионирование

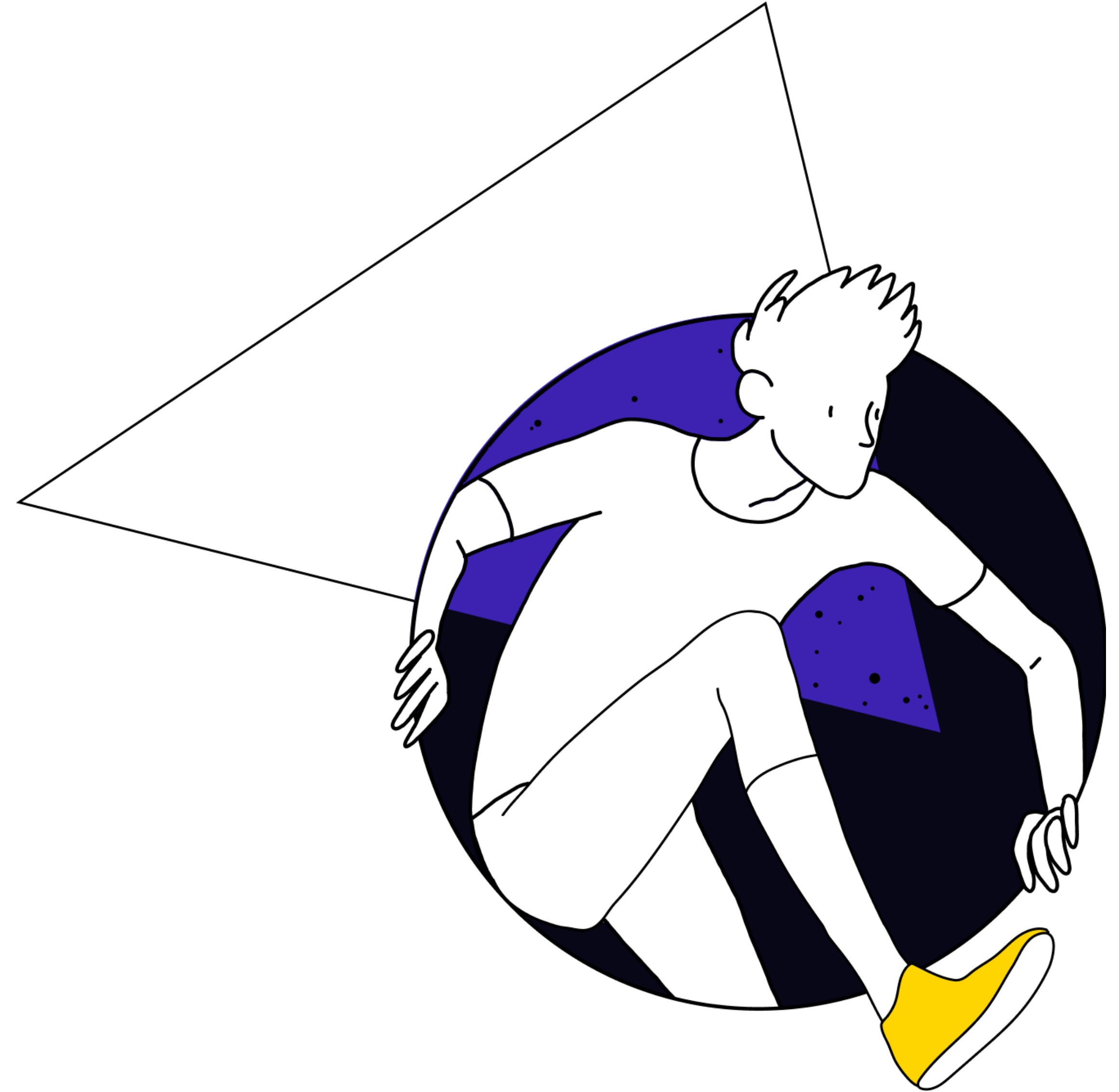
Версия эндпоинта – изменяется при обратном несовместимых изменениях в эндпоинте, при сохранении сущностей (изменения в обязательных параметрах, полях итд)

`/v1/orders/v1/create`

`/v1/cart/v2/update`

`/v2/orders/v1/create`

Вопросы?



REST

REpresentational State Transfer

- Клиент-серверная архитектура

REST

REpresentational State Transfer

- Клиент-серверная архитектура
- Отсутствие состояния

REST

REpresentational State Transfer

- Клиент-серверная архитектура
- Отсутствие состояния
- Кэширование

REST

REpresentational State Transfer

- Клиент-серверная архитектура
- Отсутствие состояния
- Кэширование
- Единообразие интерфейса

REST

REpresentational State Transfer

- Клиент-серверная архитектура
- Отсутствие состояния
- Кэширование
- Единообразие интерфейса
- Слоистая архитектура

REST

REpresentational State Transfer

- Клиент-серверная архитектура
- Отсутствие состояния
- Кэширование
- Единообразие интерфейса
- Слоистая архитектура
- Код по запросу*

Модель зрелости REST-сервисов

0: JSON over HTTP

- HTTP используется исключительно как транспорт
- Нередко всё «висит» на одном эндпоинте
- Обычно не используются никакие методы, кроме GET и POST
- Обычно не используются HTTP-коды

Пример: `POST /api/v1`

Модель зрелости REST-сервисов



0: JSON over HTTP

1: Resources

- Появляются ресурсы
- На уровне api выделяются объекты
- Разные методы начинают использоваться для работы с ресурсами, но часто всё ещё только GET и POST
- Обычно не используются HTTP-коды

Пример: POST /api/v1/order, GET /api/v1/cart

Модель зрелости REST-сервисов

0: JSON over HTTP

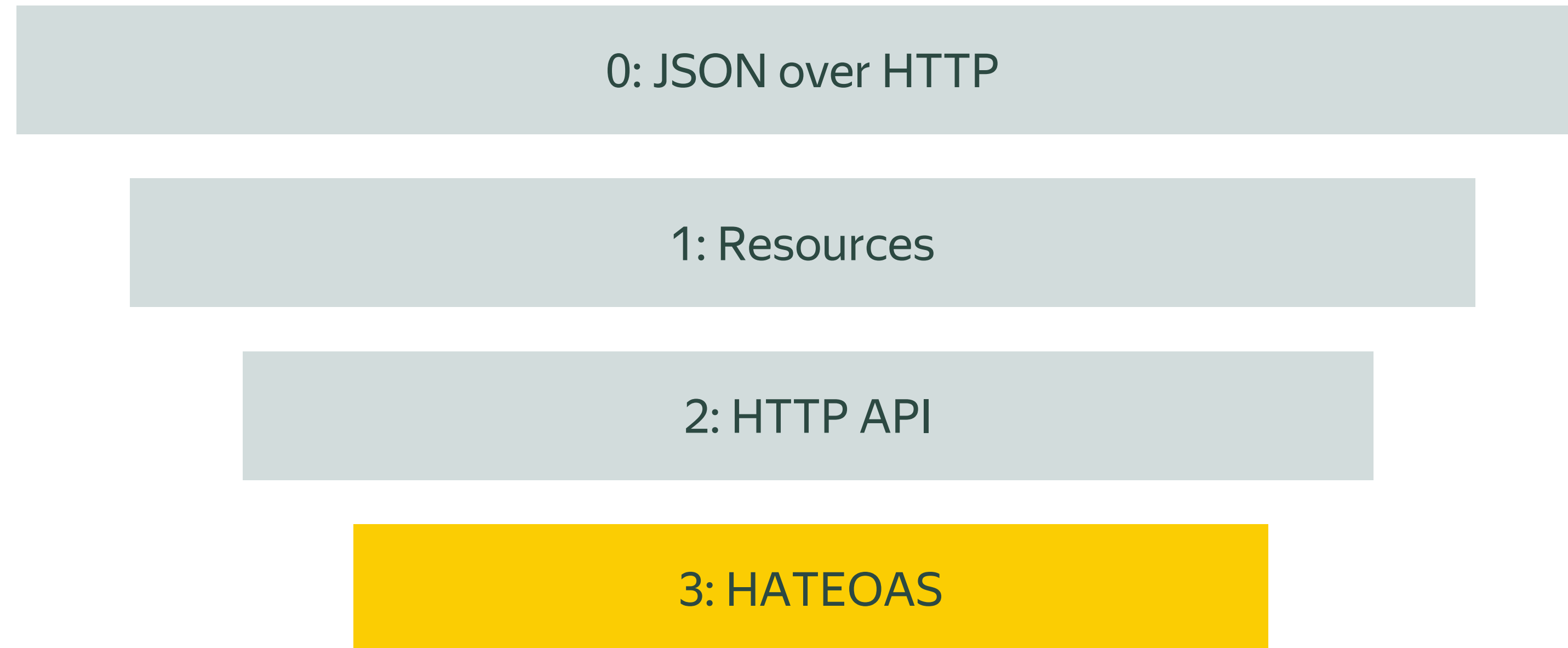
1: Resources

2: HTTP API

- Начинают правильно использоваться HTTP-методы
- Каждый метод играет особую роль в управлении ресурсами
- Начинают использоваться HTTP-коды
- Начинают использоваться HTTP-заголовки
- Чаще всего именно этот уровень называют RESTful

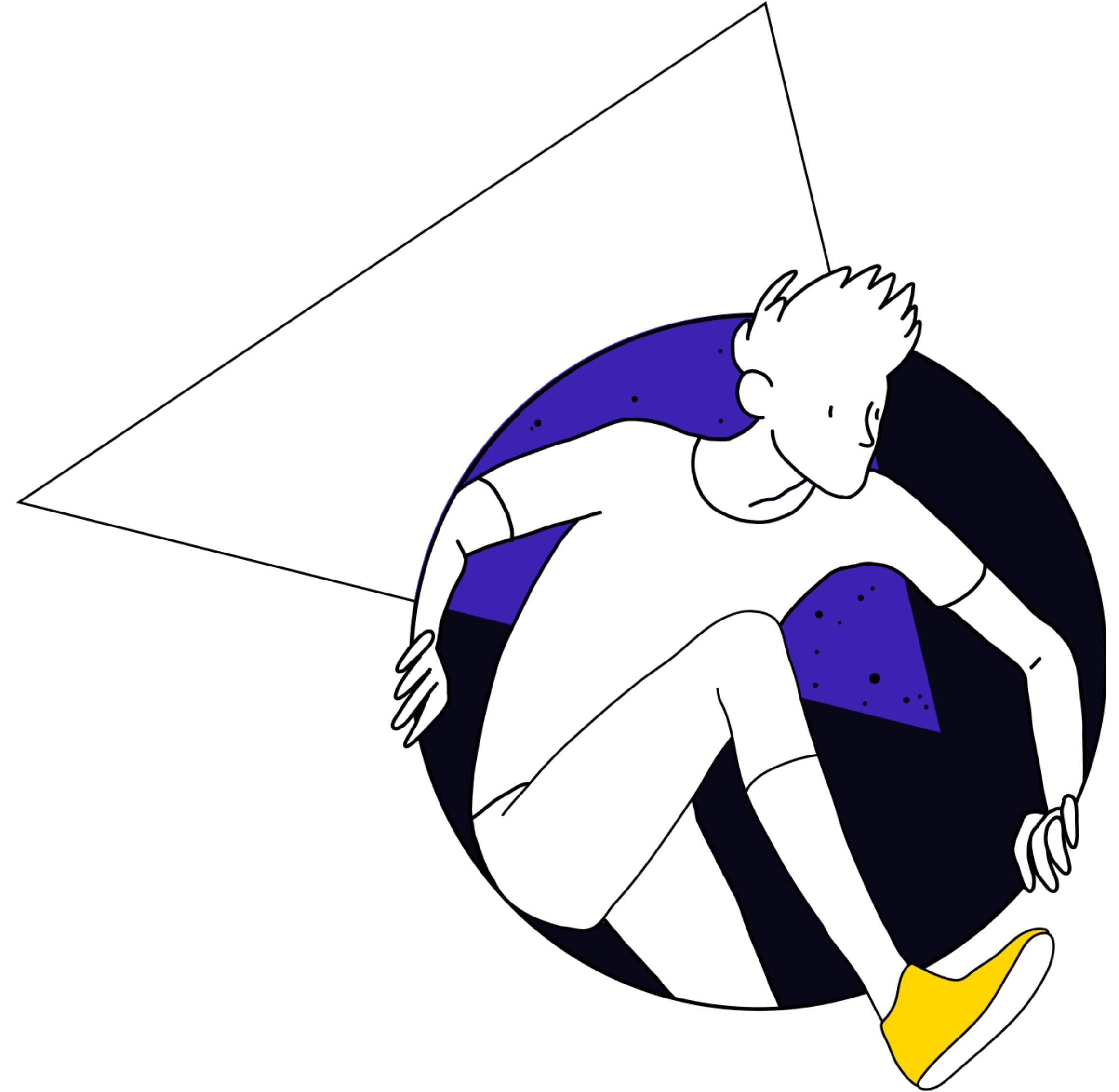
Пример: `POST /api/v1/order`, `PUT /api/v1/order`

Модель зрелости REST-сервисов



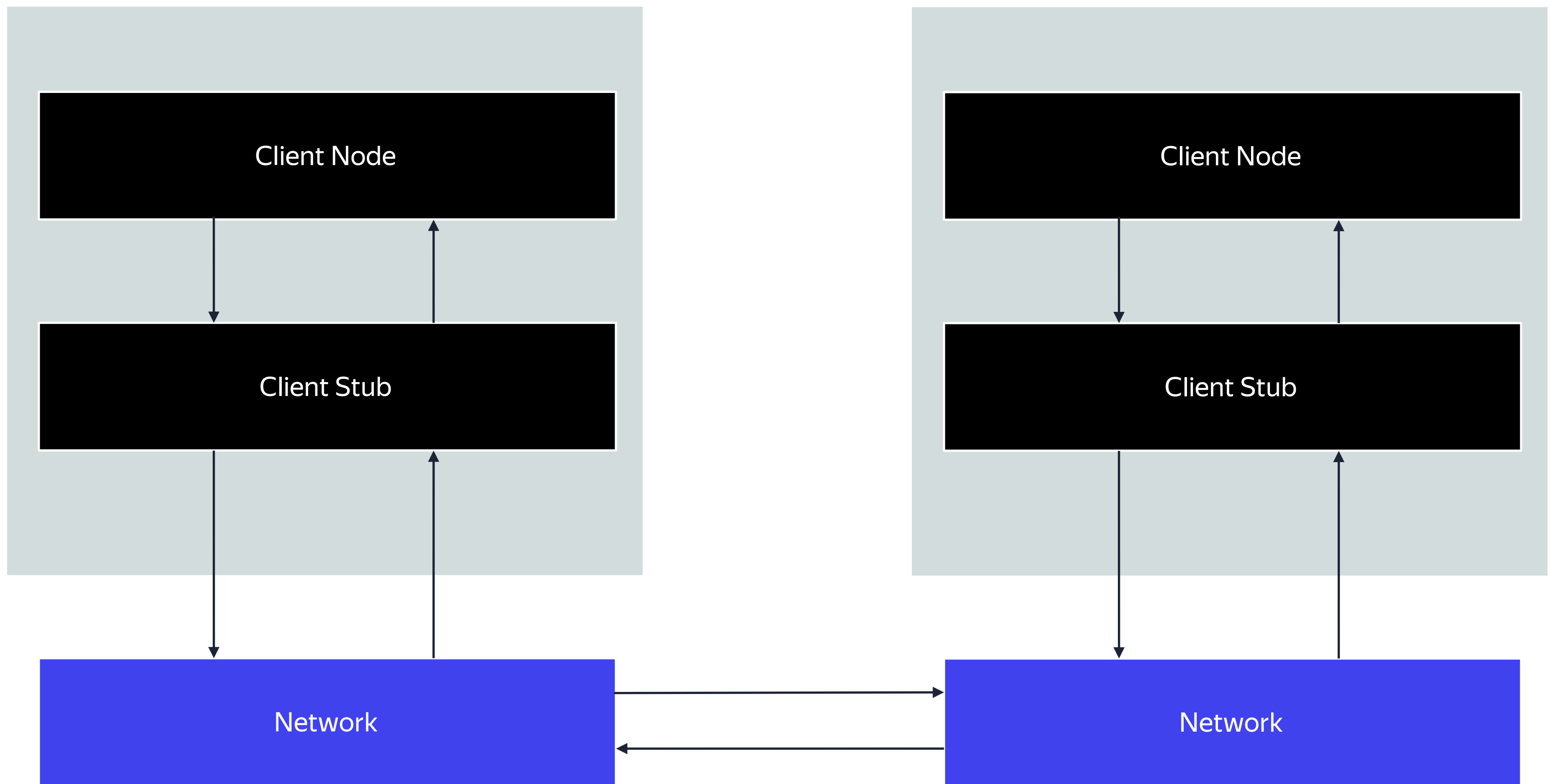
- **H**ypermedia **a**s the **E**ngine of **A**pplication **S**tate
- На этом уровне мы сообщаем потребителям о связи между ресурсами и о возможностях ресурса
- Типы ресурсов, представления и их связи стандартизированы
- Позволяет клиентам использовать API, не зная всего API заранее

Вопросы?



RPC

Remote Procedure Call



JSON-RPC

Позиционные параметры

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [23, 42],
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": -19,
  "id": 1
}
```

JSON-RPC

Позиционные параметры

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [23, 42],
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": -19,
  "id": 1
}
```

JSON-RPC

Позиционные параметры

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}
<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [23, 42],
  "id": 1
}
<--
{
  "jsonrpc": "2.0",
  "result": -19,
  "id": 1
}
```

JSON-RPC

Позиционные параметры

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}
<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [23, 42],
  "id": 1
}
<--
{
  "jsonrpc": "2.0",
  "result": -19,
  "id": 1
}
```

JSON-RPC

Позиционные параметры

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [23, 42],
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": -19,
  "id": 1
}
```

JSON-RPC

Именованные параметры

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": {
    "subtrahend": 23,
    "minuend": 42
  },
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

```
-->
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": {
    "minuend": 42,
    "subtrahend": 23
  },
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

JSON-RPC

Обработка ошибок

```
-->
{
  "jsonrpc": "2.0",
  "method": 123,
  "params": {
    "subtrahend": 23,
    "minuend": 42
  },
  "id": 1
}

<--
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32600,
    "message": "Invalid Request"
  },
  "id": null
}
```

SOAP

Simple Object Access Protocol

POST /v4/soap/ HTTP/1.1

Content-Length: 686

Content-Type: text/xml; charset=utf-8

SOAPAction: "GetClientInfo"

SOAP

Simple Object Access Protocol

POST /v4/soap/ HTTP/1.1

Content-Length: 686

Content-Type: text/xml; charset=utf-8

SOAPAction: "GetClientInfo"

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Header>
    <locale>ru</locale>
  </soap:Header>
  <soap:Body>
    <Login>admin</Login>
  </soap:Body>
</soap:Envelope>
```

SOAP

Simple Object Access Protocol

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Body>
    <Login xsi:type="xsd:string">admin</Login>
    <Email xsi:type="xsd:string">admin@yandex.ru</Email>
    <Role xsi:type="xsd:string">Admin</Role>
  </soap:Body>
</soap:Envelope>
```

gRPC

- Абстрактная спецификация

gRPC

- Абстрактная спецификация
- Поддерживает как одиночные запросы, так и стриминговые

gRPC

- Абстрактная спецификация
- Поддерживает как одиночные запросы, так и стриминговые
- Поддержка метаданных, таймаутов и отмены запросов на уровне протокола

gRPC

- Абстрактная спецификация
- Поддерживает как одиночные запросы, так и стриминговые
- Поддержка метаданных, таймаутов и отмены запросов на уровне протокола
- Описание сообщений осуществляется на некотором IDL (Interface Definition Language)

Типичная реализация gRPC

- В качестве IDL используется proto

```
service Greeter {  
  rpc SayHello (HelloRequest) returns (HelloReply) {}  
}
```

```
message HelloRequest {  
  string name = 1;  
}
```

```
message HelloReply {  
  string message = 1;  
}
```

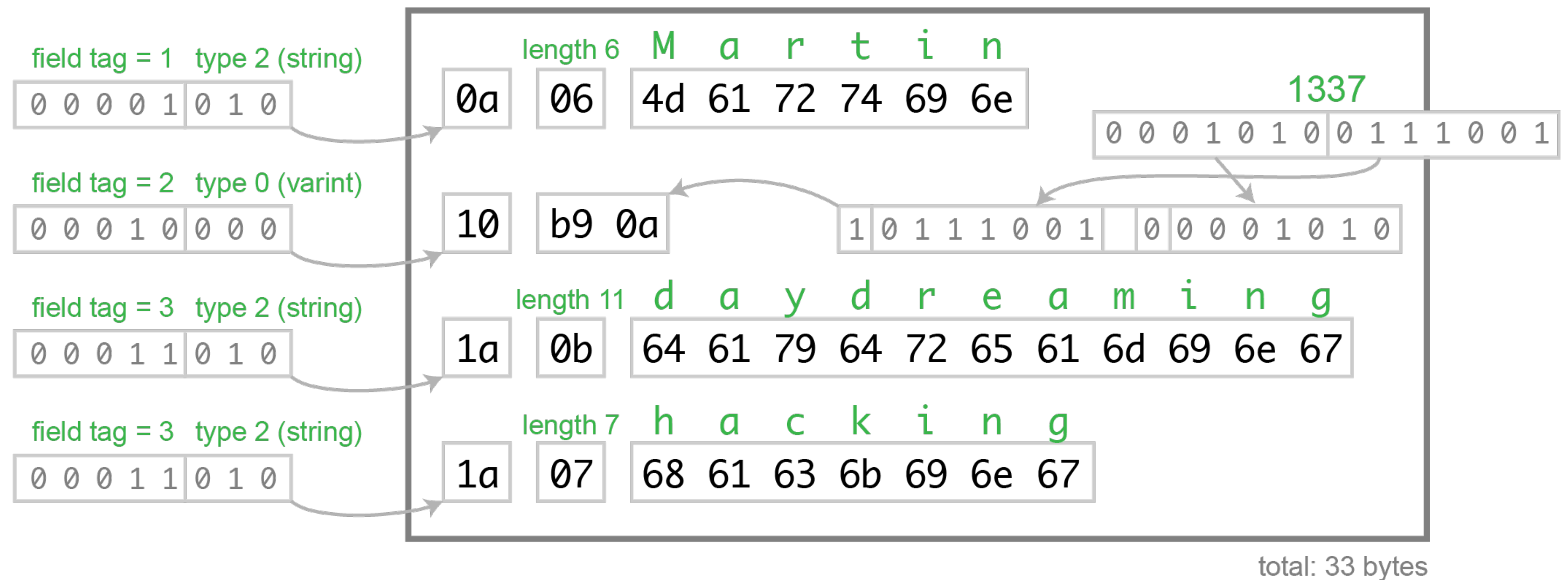
Типичная реализация gRPC

- В качестве IDL используется proto
- gRPC плагин позволяет из proto-описаний получать исходники на C++/Java/Python etc

Типичная реализация gRPC

- В качестве IDL используется proto
- gRPC плагин позволяет из proto-описаний получать исходники на C++/Java/Python etc
- Сервисы обмениваются proto-сообщениями

Protocol Buffers



Обратная совместимость

- proto задумывался с упором на обратную совместимость

Обратная совместимость

- proto задумывался с упором на обратную совместимость
- В proto3 отсутствуют обязательные поля

Обратная совместимость

- proto задумывался с упором на обратную совместимость
- В proto3 отсутствуют обязательные поля
- В proto2 и proto3 новые поля появляются прозрачно для потребителя, нужно всего лишь описать их и начать использовать

Обратная совместимость

- proto задумывался с упором на обратную совместимость
- В proto3 отсутствуют обязательные поля
- В proto2 и proto3 новые поля появляются прозрачно для потребителя, нужно всего лишь описать их и начать использовать
- Но логически обязательные поля нужно либо закрывать значениями по умолчанию, либо самостоятельно обрабатывать их отсутствие

Прозрачное управление запросом

- Кодогенерация делает невозможными опечатки или проблему с типами

Прозрачное управление запросом

- Кодогенерация делает невозможными опечатки или проблему с типами
- Запрос можно отменить как на клиенте, так и на сервере

Прозрачное управление запросом

- Кодогенерация делает невозможными опечатки или проблему с типами
- Запрос можно отменить как на клиенте, так и на сервере
- Можно выставить таймаут с обеих сторон

Прозрачное управление запросом

- Кодогенерация делает невозможными опечатки или проблему с типами
- Запрос можно отменить как на клиенте, так и на сервере
- Можно выставить таймаут с обеих сторон
- Сервер умеет обнаруживать таймаут и отменять запрос

Обработка ошибок

- В ответе присутствует машиночитаемый статус

Обработка ошибок

- В ответе присутствует машиночитаемый статус
- Есть машиночитаемый код ответа, во многом напоминающий код HTTP

Обработка ошибок

- В ответе присутствует машиночитаемый статус
- Есть машиночитаемый код ответа, во многом напоминающий код HTTP
- Есть заготовки для типичных ошибок

Обработка ошибок

- В ответе присутствует машиночитаемый статус
- Есть машиночитаемый код ответа, во многом напоминающий код HTTP
- Есть заготовки для типичных ошибок
- Передаются метаданные, в которые можно встроить информацию для трассировки

Обработка ошибок

- В ответе присутствует машиночитаемый статус
- Есть машиночитаемый код ответа, во многом напоминающий код HTTP
- Есть заготовки для типичных ошибок
- Передаются метаданные, в которые можно встроить информацию для трассировки
- Есть интерцепторы в `runtime` библиотеках

Вопросы?



Спасибо за внимание

